

Flutter Assessment & Training Portal

Master Operational Guide for Teachers, Reviewers & Administrators

1. System Overview & Architecture

The Flutter Assessment & Training Portal is an advanced, Docker-isolated web platform designed to evaluate students across progressive stages of Dart and Flutter development. It provides real-time interactive UI previews, multi-file project support, and hybrid automated/manual grading.

- **Dockerized Execution Engine:** Every student submission runs inside an isolated Docker container (flutter-runner) with pre-warmed Flutter SDKs and Linux font bundles.
- **Multi-File Workspace Support:** Students and teachers can work across multi-file project structures (e.g., lib/main.dart, lib/models/item.dart, lib/widgets/card.jsx). Code is stored and serialized as JSON file maps.
- **Real-Time Interactive Previews:** For UI challenges, the portal compiles Flutter Web bundles on the fly, rendering live interactive applications directly inside iframe viewports.
- **Extended Proxy Timeouts (300s):** The Nginx proxy layer is configured with 5-minute timeouts to ensure complex Flutter Web builds complete without dropping connections.

2. Level Creation & Progression Management

The curriculum is structured into 7 progressive stages. Students must clear earlier levels before advancing to complex state management and full-stack project builds.

- **Level 1A–1C (Dart Fundamentals):** Pure Dart, Console Output, Variables, OOP, Null Safety, Collections.
- **Level 2A–2C (Flutter Basics):** UI Construction, Scaffold, Row, Column, Stack, Images, Icons (No State/Networking).
- **Level 3A–3C (Interactive Flutter):** StatefulWidget, TextFields, Buttons, Forms, Controllers, Navigation.
- **Level 4A–4C (State Management):** Provider, Riverpod, Shared State, Lifecycle management.
- **Level 5A–5C (Networking):** HTTP, REST APIs, JSON CRUD, Mock backend integration.
- **Level 6A–6C (Storage):** Local persistence, Shared Preferences, SQLite, File handling.
- **Level 7+ (Capstone Projects):** E-commerce, Hospital App, Banking App, Chat App.

How to Create & Configure Levels:

1. Navigate to **Levels Management** in the Admin Sidebar and click **+ Add New Level**.
2. Set the **Level Code** (e.g., 3A), **Assessment Type**, and **Title**.
3. Configure **Question Count** (number of problems assigned per session) and **Duration Minutes**.
4. Set the **Pass Threshold (%)** required to unlock the next level (default: 70% or 80%). Click **Save Level**.

3. Question Bank & Problem Management (In-Depth Guide)

The **Question Bank** (AdminQuestions_new.jsx) is where teachers design interactive challenges. Here is how to utilize all advanced capabilities:

- A. Multi-File Workspace & Starter Code:** In the Starter Code section, define boilerplate for lib/main.dart. Click **+ Add File** to create supplementary files (e.g., lib/models/user.dart), teaching modular architecture.
- B. Resource Bundles (Images, Fonts, JSON):** In the Resources tab, upload PNGs, custom TTF fonts, or datasets. The Docker runner automatically copies them to assets/images/ inside the container and auto-registers them in pubspec.yaml!
- C. Third-Party Package Injection:** In the Required Packages section, select checkboxes for libraries like provider, http, shared_preferences, google_fonts, or fl_chart. They are automatically injected into pubspec.yaml during compilation.
- D. Mock APIs & Backend Integration (Level 5+):** For networking problems without internet access, set a **Mock API Route** (e.g., /api/mock/products) and paste the exact JSON response payload. The container intercepts HTTP calls and returns this mock data!
- E. Mock Database Seeding (Level 6+):** Provide seed SQL queries or initial JSON data in the **Mock DB Seed** box to prepopulate databases before student code executes.
- F. Custom Test Code (Widget Verification):** Write automated verification suites using Flutter's testWidgets library to assert button taps, text renders, and state transitions.

4. Comprehensive Bulk Upload Guide (CSV Import)

To import dozens of questions simultaneously, use the **Bulk Upload / CSV Import** feature in the Question Bank.

1. Save your spreadsheet as a **UTF-8 Encoded CSV** (.csv).
2. Click **Bulk Upload / CSV Import** -> **Choose File** -> **Upload & Validate**.
3. Review the validation preview table (validating level codes and JSON syntax), then click **Confirm Import**.

Required CSV Headers & Data Formats:

Header	Req?	Format / Example
level	YES	Level code matching DB (e.g., 1A, 2B, 3C)
title	YES	Question title (e.g., <i>Login Form Validation</i>)
description	YES	Detailed Markdown instructions & rules
starter_code	YES	Dart string OR JSON object map: { <code>"lib/main.dart": "import..."</code> }
ui_required_widgets	NO	JSON Array: [<code>"Scaffold", "AppBar", "ListView"</code>]
required_packages	NO	JSON Array: [<code>"provider", "http", "google_fonts"</code>]
mock_api_route	NO	Endpoint string: <code>/api/mock/users</code>
mock_api_response	NO	JSON payload: { <code>"users": [{"id": 1}]</code> }
custom_test_code	NO	Complete testWidgets Dart test suite

■ **CRITICAL TIP FOR CSV MULTI-LINE CODE:** When writing code snippets or JSON maps inside spreadsheet cells (Excel/Numbers/Google Sheets), wrap the entire cell contents in double quotes " and press Alt+Enter (Windows) or Option+Return (Mac) for line breaks. Do not use raw unescaped quotes inside JSON strings!

5. Test Slot & Schedule Configuration

To administer an exam, navigate to **Test Schedules / Slots** (AdminTestSlots.jsx) and click **+ Create Schedule**.

- **Schedule Details:** Set Schedule Name (e.g., *Batch 2026 Mid-Term*), Start/End Date & Time, and Duration Minutes.
- **Staff Assignment:** Assign specific staff members to three distinct operational roles:
 - **Live Teacher:** Responsible for live classroom supervision and addressing student queries.
 - **Code Reviewer Teacher:** Assigned to evaluate algorithm/logic tests (AdminSubmissions.jsx).
 - **UI Reviewer Teacher:** Assigned to evaluate interactive Flutter UI tests (AdminManualGrading.jsx).

Once created, registered students will see the assessment unlock on their dashboard at the designated start time.

6. Student Portal Rules & Execution Environment

When students start an assessment, they enter a monitored, high-performance coding environment tailored for Flutter development.

- **Dark Monaco Editor:** Configured strictly in Dark Theme (vs-dark) for optimal syntax highlighting and readability.

- **Interactive Live Preview:** Clicking **Run Preview** compiles code server-side and opens an interactive HTML5 viewport on the right half of the screen. Students can click buttons, type text, scroll lists, and test animations in real-time!
- **Submission Mechanics:** Clicking **Submit** records the multi-file code map and freezes the score. If a student disconnects, session timers persist server-side while code drafts are restored from browser storage upon reconnection.

7. Grading & Result Analysis (With Excel Export)

Grading combines **Automated AI/Test Assertions (50%)** and **Teacher Manual Review (50%)**. Both code review pages now feature instant searchable filtering and one-click Excel (.xlsx) spreadsheet exporting!

A. Coding Test Code Review (AdminSubmissions.jsx):

- **Searchable Filtering:** Use the search bar to instantly search across Student Name, Email, Roll Number, Problem Title, or Slot Name.
- **Status Filter:** Filter submissions by ALL, PASS, or FAIL.
- **Export XLSX Spreadsheet:** Click the green **Export XLSX** button. Instantly downloads an Excel sheet containing: S.No, Submission ID, Student Name, Roll Number, Email, Test Slot, Problem ID, Status, Auto Score (%), Match Percent, Code Length, and Timestamp!

B. UI Test Code Review (AdminManualGrading.jsx):

- **Searchable Filtering:** Use the newly added search input bar to instantly filter UI submissions by Student Name, Roll Number, Email, Problem Title, or Slot Name.
- **Live Interactive Preview Review:** Select any student submission and click the **Live Preview & Mockup** tab. Teachers can interact with the student's compiled Flutter web app live inside the iframe to check animations, responsiveness, and button behaviors!
- **Student Code Inspection:** Click the **Student Code** tab to inspect all submitted files (lib/main.dart, models, widgets).
- **Assigning Manual Grades & Feedback:** Enter a Manual Score (0–100) and constructive feedback. The system auto-computes the final score:

$$\text{Final Score} = (\text{Automated Score} \times 0.5) + (\text{Manual Score} \times 0.5)$$
- **Export XLSX Spreadsheet:** Click the green **Export XLSX** button. Instantly downloads an Excel sheet containing: S.No, Submission ID, Student Name, Email, Roll Number, Test Slot, Problem Title, Auto Score (50%), Manual Score (50%), Final Combined Score (%), Teacher Feedback, Status, and Code Files Count!